COMPRAS · TIENDA EN LÍNEA

Plan — Tienda en línea de productos

Plan de implementación para una tienda de productos con carrito, checkout, entrega a domicilio y seguimiento, sin modificar el backend actual.

Proyecto	Compras-Frontend-Cliente
Backend	Compras-Backend (Spring Boot 4 + PostgreSQL)
Fecha	22 de octubre de 2026
Estado	Propuesta de implementación
Alcance	MVP de tienda en línea con datos temporales

> **Documento de planificación.** No implementa código, no modifica el backend y no crea entidades nuevas. Su objetivo es ordenar el trabajo y dejar explícitas las limitaciones antes de programar.

Contenido

1. 0. Cómo leer este documento
2. 1. Resumen ejecutivo
3. 2. Decisiones tomadas
4. 3. Alcance
5. 4. Estado actual y brecha
6. 5. Arquitectura propuesta
7. 6. Contrato con el backend
8. 7. Modelo de datos del storefront
9. 8. Flujos
10. 9. Endpoints del BFF
11. 10. Almacén temporal de pedidos
12. 11. Rutas y pantallas
13. 12. Diseño visual
14. 13. SEO y metadata
15. 14. Seguridad
16. 15. Pruebas
17. 16. Fases de implementación
18. 17. Archivos previstos
19. 18. Tarjetas de prueba
20. 19. Riesgos y mitigaciones
21. 20. Limitaciones que se deben decir al usuario
22. 21. Migración futura al backend de tienda
23. 22. Decisiones abiertas
24. 23. Checklist de aceptación
25. 24. Glosario
26. 25. Referencias internas
27. 26. Primer commit recomendado
28. 27. Conclusión

Proyecto: Compras-Frontend-Cliente **Backend:** Compras-Backend (Spring Boot 4 + PostgreSQL) **Fecha:** 2026-10-22 **Estado:** Propuesta de implementación. **No hay código implementado todavía. Objetivo:** convertir el frontend Next.js en una tienda funcional de productos de despensa, con carrito, checkout, entrega a domicilio, pago simulado y seguimiento de pedidos, **sin modificar el backend actual**.

---

0. Cómo leer este documento

Este plan es para **este repositorio** y para el backend **tal como está hoy**. No es el plan de restaurante.

El archivo `docs/PLAN.md` de este mismo frontend es una copia vieja del plan del backend de restaurante. Se deja como referencia histórica, pero **no se debe seguir**: menciona platillos, recetas, insumos, WebSocket y `/api/tienda`, cosas que todavía no existen. El plan vigente para este frontend es este archivo.

La regla principal del proyecto es:

> **El backend no se toca en esta etapa.** Todo lo que falta se resuelve en el frontend con un BFF, datos temporales y una interfaz honesta sobre lo que el backend sí puede hacer.

---

1. Resumen ejecutivo

Vamos a construir una tienda de productos de despensa encima del POS actual. El backend ya sabe vender, descontar stock, cobrar en efectivo, cobrar con tarjeta simulada y llevar el historial de ventas. Lo que no tiene es una cara pública para el cliente.

La solución es un **BFF (Backend For Frontend)** dentro de Next.js:

- El navegador habla solo con `/api/bff/*`, en el mismo origen.
- El servidor de Next habla con `http://localhost:8081/api/local/*` usando un **usuario técnico** con permisos mínimos.
- El JWT nunca llega al navegador.
- Los precios, el stock y el total se recalculan en el servidor; el navegador no es la fuente de verdad.
- Los datos que el backend no tiene (dirección, teléfono, tipo de entrega, notas) se guardan en un almacén temporal de pedidos, separado de la venta real.

Lo que el cliente podrá hacer

- Ver el catálogo de productos activos.
- Navegar por categorías.
- Buscar productos por nombre.

- Ver el detalle de un producto.
- Agregar productos al carrito.
- Cambiar cantidades y eliminar productos.
- Elegir entrega a domicilio o recoger en tienda.
- Dejar sus datos de contacto y dirección.
- Pagar en efectivo contra entrega o con tarjeta de prueba.
- Confirmar el pedido y recibir un número de pedido.
- Consultar sus pedidos en este navegador.
- Ver el estado del pedido según la venta real del backend.
- Volver a pedir un pedido anterior.

Lo que el negocio podrá hacer

- Ver la venta online en el mismo historial del POS de Angular.
- Ver el stock descontado.
- Confirmar la venta desde Angular cuando el pedido salga.
- Anular la venta desde Angular si algo salió mal.
- Usar el mismo catálogo, las mismas categorías y las mismas imágenes del POS.

Lo que NO se resolverá en esta etapa

- No habrá cuentas de cliente ni login real de cliente.
- No habrá una entidad `Pedido` en el backend.
- No habrá estados de cocina, "en camino" ni seguimiento en tiempo real.
- No habrá costo de envío, zonas de reparto ni horarios.
- No habrá pago real con tarjeta ni pasarela.
- No habrá cupones, descuentos ni impuestos desglosados.
- No habrá notificaciones por correo, SMS o WhatsApp.
- No habrá panel administrativo de pedidos en producción; solo uno interno opcional para la demo.

Criterio de éxito de la integración

La meta no es "que se vea bonito". La meta es que este flujo funcione de punta a punta:

1. Hay una caja abierta en el POS.
2. El cliente agrega productos al carrito.
3. El cliente confirma un pedido en Next.js.
4. Se crea una venta real en el backend.
5. El stock baja.
6. La venta aparece en Angular.

7. El pedido aparece en `/pedidos` de Next.js.
8. El POS confirma la venta y Next.js muestra el pedido como entregado o recogido.
9. Si el POS anula la venta, Next.js muestra el pedido como cancelado.
10. Un doble clic o un reintento no crea dos ventas.

---

2. Decisiones tomadas

#	Decisión	Elección	Por qué	Consecuencia
1	¿Cómo habla el navegador con el backend?	BFF en Next.js	El backend exige JWT de empleado y no tiene endpoints públicos de tienda. Poner el token en el navegador sería expuesto y frágil.	El navegador solo conoce `/api/bff/*`.
2	¿Qué usuario usa el BFF?	Usuario técnico `TIENDA`	Permite leer catálogo y crear ventas sin dar permisos de administración.	Las ventas online quedan a nombre de ese usuario. Es una limitación de auditoría.
3	¿Dónde viven los datos del pedido que el backend no tiene?	Almacén temporal en el servidor de Next (`OrderStore`)	El backend no tiene cliente, dirección ni tipo de entrega. No se puede inventar una tabla.	Se usa un archivo JSON local para la demo, detrás de una interfaz reemplazable.
4	¿El cliente inicia sesión?	No. Checkout como invitado	No existe autenticación de cliente. Un login falso sería peor que no tenerlo.	El historial de pedidos se vincula a una cookie anónima del navegador.
5	¿Entrega o recoger?	Ambos, sin costo de envío	El negocio pidió entrega a domicilio. El backend no cobra envío.	Se muestra "Envío sin costo (demo)" y se aclara que el costo real no está implementado.
6	¿Pago real?	Efectivo demo y tarjeta simulada	El backend solo tiene terminal simulada o física. No hay pasarela.	Se usan tarjetas de prueba. Nunca se piden datos reales.
7	¿Estados del pedido?	Derivados de `confirmed`, `cancelled` y `paymentStatus`	El backend no tiene estados de pedido.	La UI dice "Estado estimado" y no inventa "en camino".
8	¿Dónde va este plan?	`PLAN.md` en la raíz del frontend	El `docs/PLAN.md` existente está obsoleto y confunde.	Este archivo es la referencia vigente.
9	¿Middleware o proxy?	`proxy.ts` si hace falta lógica por request	Next.js 16 renombró `middleware` a `proxy`.	No se usará `middleware.ts`. Para cabeceras simples basta `next.config.ts`.
10	¿Se modifica el backend?	No	Es una restricción explícita.	Las limitaciones se documentan y se resuelven en el frontend.

---

3. Alcance

3.1 Incluido en el MVP

- Home con catálogo y categorías.
- Página de categoría.
- Página de detalle de producto.
- Buscador.
- Carrito persistente en el navegador.
- Checkout con entrega o recogida.
- Validación de datos en cliente y servidor.
- Creación de venta real en el backend.
- Pago en efectivo demo.
- Pago con tarjeta simulada.
- Idempotencia del pedido.
- Confirmación de pedido.
- Historial de pedidos por sesión anónima.
- Detalle de pedido con estado derivado.
- Actualización por polling.
- Volver a pedir.
- Diseño responsive.
- SEO básico.
- Accesibilidad básica.
- Pruebas unitarias de la lógica crítica.
- Prueba manual de integración con Angular.
- Documentación de limitaciones y migración futura.

3.2 Excluido a propósito

- Registro y login de clientes.
- Recuperación de contraseña de cliente.
- Perfil de cliente.
- Direcciones guardadas.
- Múltiples direcciones.
- Zonas de reparto.
- Costo de envío.
- Horarios de atención.
- Cupones.

- Puntos de fidelidad.
- Reseñas.
- Inventario por sucursal.
- Notificaciones.
- Chat.
- Mapa.
- Facturación.
- Devoluciones en línea.
- Cancelación en línea por parte del cliente.
- Panel de administración completo.
- WebSocket.
- Stripe, MercadoPago o terminal física real.

3.3 Supuestos

- El backend corre en `http://localhost:8081`.
- PostgreSQL está disponible y migrado.
- Hay productos activos con precio, stock e imagen.
- Hay categorías creadas.
- Hay una caja abierta antes de probar el checkout.
- El frontend Angular corre en `http://localhost:4200`.
- El frontend Next.js corre en `http://localhost:3000`.
- Node.js 20 o superior.
- Se usa español de México y pesos mexicanos (MXN).
- La tienda es de un solo local.
- No se requiere autenticación de cliente.
- El entorno de demo es local, no producción.

---

4. Estado actual y brecha

4.1 Backend actual

Área	Lo que existe	Lo que falta para la tienda
Productos	CRUD, listado de activos, búsqueda, categoría, imagen	No hay `GET /products/{id}`; no hay endpoint público
Categorías	CRUD y listado	No hay endpoint público
Ventas	Crear, listar, detalle, confirmar, anular	No hay cliente, pedido, entrega ni estado de pedido

Área	Lo que existe	Lo que falta para la tienda
Pagos	Efectivo, tarjeta simulada, reversa	No hay pasarela real ni pago contra entrega real
Caja	Abrir, cerrar, activa, historial	Obligatoria para vender
Seguridad	JWT de empleado y permisos	No hay JWT de cliente
Prefijos	`/api/local/**` y `/api/uploads/**`	No existe `/api/tienda/**`

4.2 Frontend Next actual

- Next.js `16.3.6`.
- React `19.2.8`.
- Tailwind CSS 4.
- TypeScript estricto.
- Sin framework de tests.
- Sin librería de componentes.
- Sin estado global.
- Sin cliente HTTP.
- Rutas existentes como stubs: `/carrito`, `/categoria`, `/checkout`, `/login`, `/pedidos`, `/platillo`, `/registro`.
- `app/pedidos/pageId.tsx` no es una ruta válida de Next.js; hay que reemplazarlo por `app/pedidos/[id]/page.tsx`.
- `app/layout.tsx` usa `LayoutProps<"/">` generado por Next.
- `next.config.ts` está vacío.
- `AGENTS.md` exige leer la documentación local de Next 16 antes de programar.

4.3 Brechas y cómo se resuelven

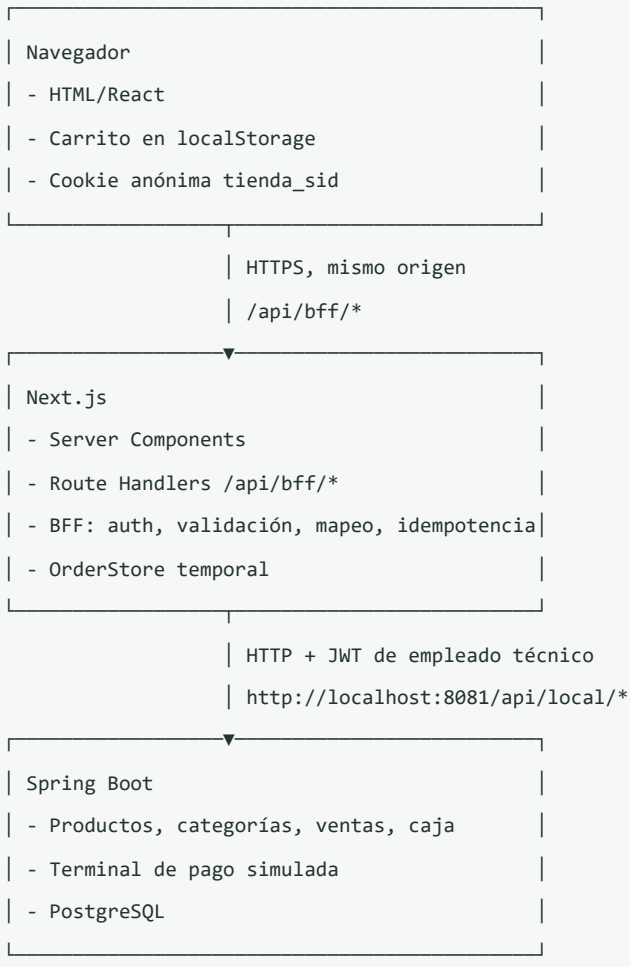
Brecha	Impacto	Solución en este plan	Riesgo
No hay catálogo público	El navegador no puede leer productos sin JWT	BFF con usuario técnico	El token debe protegerse
No hay `GET /products/{id}`	No se puede pedir un producto por id	El BFF descarga el catálogo y filtra por id	Costoso con catálogos grandes; aceptable en demo
No hay clientes	No hay login ni historial por usuario	Checkout invitado y cookie anónima	El historial se pierde si se borra la cookie
No hay pedidos	No hay dirección ni tipo de entrega	`OrderStore` temporal	No es durable ni multiusuario
No hay estados de pedido	No se puede mostrar "en camino"	Derivar de la venta real	La UI debe ser honesta
No hay costo de envío	El total no incluye envío	Mostrar "Envío sin costo (demo)"	No sirve para producción
No hay pago real	No se puede cobrar con tarjeta real	Tarjetas de prueba	No usar datos reales

Brecha	Impacto	Solución en este plan	Riesgo
Caja obligatoria	Sin caja no se puede vender	Bloquear checkout y mostrar tienda cerrada	Depende del POS
Stock no atómico	Dos pedidos pueden sobrevender	Revalidar y mostrar error	No elimina la carrera
Usuario técnico compartido	Las ventas online no distinguen clientes	Documentarlo	Auditoría limitada
Token de 5 horas	El BFF puede quedarse sin sesión	Cache y re-login automático	Depende del backend
Imágenes externas	`next/image` requiere configuración	`remotePatterns`	Si cambia el host, hay que actualizar
Next 16 cambió APIs	`params`, `cookies`, `proxy`	Leer docs y usar APIs nuevas	Código viejo no compila

---

5. Arquitectura propuesta

5.1 Diagrama



5.2 Componentes

Componente	Responsabilidad
`app/api/bff/catalogo`	Lista productos y categorías para la tienda
`app/api/bff/productos/[id]`	Detalle de producto, obtenido del catálogo
`app/api/bff/categorias`	Lista de categorías
`app/api/bff/estado-tienda`	Indica si hay caja abierta
`app/api/bff/pedidos`	Crea un pedido y lista los de la sesión
`app/api/bff/pedidos/[id]`	Detalle de pedido y estado actualizado
`lib/server/backend`	Cliente HTTP del backend, auth y errores
`lib/server/orders`	Almacén temporal de metadatos
`lib/server/session`	Cookie anónima de pedidos
`lib/shared`	Tipos, dinero, validación, slugs, estados
`components`	UI de catálogo, carrito, checkout y pedidos

5.3 Por qué un BFF y no llamadas directas

- El backend no tiene endpoints públicos de tienda.
- El JWT de empleado no puede viajar al navegador.
- El backend devuelve campos internos como `cost`, `active` y `hasHistory`; el BFF los elimina.
- El backend usa nombres y formatos pensados para el POS; el BFF los adapta a la tienda.
- El BFF puede validar el carrito y recalcular el total antes de crear la venta.
- El BFF puede guardar los metadatos que el backend no tiene.
- El BFF puede cambiar de adaptador cuando exista `/api/tienda` sin rehacer la UI.
- El BFF puede centralizar el manejo de errores y evitar mostrar mensajes técnicos.

5.4 Por qué no usar el navegador contra `/api/local`

- Expondría el token.
- Obligaría a habilitar CORS para el dominio de la tienda.
- Permitiría que un usuario manipule precios o cantidades.
- Expondría `cost` y otros datos internos.
- Imposibilitaría guardar metadatos y validar en servidor.
- Haría imposible cambiar de backend sin tocar todos los componentes.

5.5 Usuario técnico de la tienda

Se crea un rol y un usuario exclusivos para el BFF.

Rol: `TIENDA`

Permiso	Uso
`VER_PRODUCTOS`	Leer catálogo
`VER_CATEGORIAS`	Leer categorías
`CREAR_VENTAS`	Crear la venta del pedido
`VER_VENTAS`	Consultar el estado del pedido
`VER_CAJA`	Saber si hay caja abierta
`CONFIRMAR_VENTAS`	Opcional, solo para panel interno de demo

No se le deben dar:

- `EDITAR_PRODUCTOS`
- `ELIMINAR_PRODUCTOS`
- `DESACTIVAR_PRODUCTOS`
- `CREAR_COMPRAS`
- `CANCELAR_VENTAS`
- `CERRAR_CAJA`
- `ABRIR_CAJA`
- permisos de usuarios, roles o reportes

Usuario sugerido: `tienda@compras.local` **Contraseña:** definida en `.env.local`, nunca en el repositorio.

Creación: desde la pantalla de usuarios y roles del Angular, o por API con un usuario administrador. El plan no crea el usuario automáticamente porque el backend no tiene bootstrap para `TIENDA`.

Consecuencia: todas las ventas online quedan registradas a nombre de `tienda@compras.local`. No se puede distinguir qué cliente hizo cada pedido desde el POS. Es una limitación aceptada para la demo.

5.6 Token del backend

- El BFF hace `POST /api/local/auth/login` con el usuario técnico.
- Guarda el token en memoria del proceso.
- Calcula una expiración conservadora: 4 horas 45 minutos, aunque el JWT dure 5 horas.
- Si no hay token o está vencido, hace login.
- Si una llamada devuelve 401, limpia el token, hace login una vez y reintenta.
- Si el login falla, devuelve un error de configuración, no un error de usuario.
- El token no se escribe en logs, archivos ni respuestas.

5.7 Variables de entorno

Server-only:

Variable	Obligatoria	Ejemplo	Uso
`BACKEND_API_URL`	Sí	`http://localhost:8081`	Base del backend
`TIENDA_API_EMAIL`	Sí	`tienda@compras.local`	Login técnico
`TIENDA_API_PASSWORD`	Sí	`***`	Login técnico
`TIENDA_ADMIN_TOKEN`	No	`***`	Panel interno opcional
`TIENDA_SESSION_SECRET`	No	`***`	Firmar cookie de sesión si se refuerza
`ORDER_STORE_PATH`	No	`.data/pedidos.json`	Archivo de pedidos
`NEXT_PUBLIC_SITE_URL`	No	`http://localhost:3000`	Enlaces y metadata

Regla: ningún secreto puede llevar el prefijo `NEXT_PUBLIC_`. Todo lo que empieza con `NEXT_PUBLIC_` se incluye en el JavaScript del navegador.

5.8 Prerrequisitos del backend

Antes de programar la tienda:

1. Arrancar PostgreSQL.
2. Arrancar el backend en `8081`.
3. Arrancar Angular en `4200`.
4. Verificar que hay productos activos.
5. Verificar que hay categorías.
6. Verificar que las imágenes cargan.
7. Crear el rol `TIENDA`.
8. Crear el usuario técnico.
9. Abrir una caja.
10. Confirmar que `POST /api/local/auth/login` responde con token.
11. Confirmar que el usuario técnico puede leer productos y crear una venta de prueba.
12. Anular la venta de prueba para no dejar basura.

5.9 Configuración de Next.js

- `next.config.ts` debe declarar `images.remotePatterns` para el host del backend.
- Si el backend está en `localhost:8081`, se permite ese host y puerto.
- Si se despliega, se agrega el dominio HTTPS real.
- No se activa `cacheComponents` en la primera fase. El modelo de caché por defecto es suficiente.
- No se usa `middleware.ts`; si hace falta lógica por request, se usa `proxy.ts`.
- Las rutas dinámicas usan `params` como `Promise` y se resuelven con `await`.
- Las cookies se leen con `await cookies()`.

---

6. Contrato con el backend

6.1 Endpoints que se usarán

Endpoint	Método	Permiso	Uso en la tienda
`/api/local/auth/login`	POST	Público	Login del usuario técnico
`/api/local/products`	GET	`VER_PRODUCTOS`	Catálogo
`/api/local/products?category=...`	GET	`VER_PRODUCTOS`	Productos por categoría
`/api/local/products/search?q=...`	GET	`VER_PRODUCTOS`	Buscador
`/api/local/categories`	GET	`VER_CATEGORIAS`	Categorías
`/api/local/cash/active`	GET	`VER_CAJA`	Saber si la tienda puede vender
`/api/local/sales`	POST	`CREAR_VENTAS`	Crear el pedido
`/api/local/sales/{id}`	GET	`VER_VENTAS`	Estado del pedido
`/api/local/sales`	GET	`VER_VENTAS`	Reconciliación opcional
`/api/local/sales/{id}/confirm`	PATCH	`CONFIRMAR_VENTAS`	Panel interno opcional
`/api/local/sales/{id}/cancel`	PATCH	`CANCELAR_VENTAS`	Solo desde Angular

6.2 Lo que se debe ocultar del cliente

El `ProductDto` del backend trae más de lo que la tienda necesita. El BFF debe devolver solo:

- `id`
- `name`
- `price`
- `stock`
- `categoryId`
- `categoryName`
- `img`

No se debe enviar al navegador:

- `cost`
- `active`
- `hasHistory`
- `barcode`
- `sku`
- cualquier dato interno de inventario o margen

6.3 Crear una venta en efectivo

```
{
  "paymentMethod": "CASH",
  "cashReceived": 250.00,
  "items": [
    { "productId": 1, "quantity": 2 }
  ],
  "idempotencyKey": "0f8fad5b-d9cb-469f-a165-70867728950e"
}
```

Notas:

- `cashReceived` debe ser igual al total calculado por el servidor en la demo.
- El backend calcula `changeAmount`.
- El backend descuenta stock solo si el pago queda `APPROVED`.
- Si no hay caja abierta, devuelve error `SALE_ERROR` con “No existe una caja abierta”.

6.4 Crear una venta con tarjeta simulada

```
{
  "paymentMethod": "DEBIT",
  "items": [
    { "productId": 1, "quantity": 1 }
  ],
  "cardPayment": {
    "cardNumber": "4111111111111111",
    "pin": "1234"
  },
  "idempotencyKey": "0f8fad5b-d9cb-469f-a165-70867728950e"
}
```

Notas:

- El BFF no envía `saleId`; el backend lo asigna.
- El backend sobrescribe `cardPayment.paymentMethod` con el método de la venta.
- Si la tarjeta se rechaza, el backend lanza una excepción y la transacción revierte: no debe quedar venta.
- Nunca se deben usar datos de tarjeta reales.

6.5 Respuesta de venta

```
{
  "saleId": 123,
```

```
  "total": 250.00,
  "paymentMethod": "CASH",
  "cashReceived": 250.00,
  "changeAmount": 0.00,
  "paymentStatus": "APPROVED",
  "lastFourDigits": null,
  "authorizationCode": null,
  "cardPaymentResponse": null
}
```

El BFF debe devolver al navegador solo lo necesario:

```
{
  "publicId": "PED-20261022-A1B2C3",
  "saleId": 123,
  "total": 250.00,
  "paymentMethod": "CASH",
  "paymentStatus": "APPROVED",
  "createdAt": "2026-10-22T14:30:00"
}
```

6.6 Detalle de venta

`GET /api/local/sales/{id}` devuelve:

- `saleId`
- `saleDate`
- `total`
- `paymentMethod`
- `items`
- `paymentStatus`
- `confirmed`
- `confirmedAt`
- `cancelled`
- `cancelledAt`
- `userId`
- `userName`

Cada item trae:

- `product`
- `quantity`

- `unitPrice`
- `unitCost`
- `subtotal`

El BFF debe eliminar `unitCost` antes de enviar el detalle al navegador. El costo es información interna.

6.7 Mapeo de errores

Situación	Backend	BFF	Mensaje al cliente
Sin caja abierta	`400 SALE_ERROR`	`503 TIENDA_CERRADA`	"La tienda no está recibiendo pedidos en este momento."
Stock insuficiente	`400 SALE_ERROR`	`409 STOCK_INSUFICIENTE`	"Un producto ya no tiene existencias suficientes."
Producto inexistente	`400 SALE_ERROR`	`409 PRODUCTO_NO_DISPONIBLE`	"Un producto de tu carrito ya no está disponible."
Tarjeta rechazada	`PAYMENT_ERROR`	`402 PAGO_RECHAZADO`	"El pago no fue aprobado. Prueba otra tarjeta de prueba."
Token inválido	`401`	`503 ERROR_BACKEND`	"No pudimos conectar con la tienda."
Sin permiso	`403`	`503 ERROR_BACKEND`	"La tienda no está configurada correctamente."
Backend caído	Error de red	`503 ERROR_BACKEND`	"No pudimos conectar con la tienda."
Datos inválidos	—	`400 DATOS_INVALIDOS`	Mensaje por campo
Doble envío	—	`200` con el mismo pedido	"Tu pedido ya estaba registrado."

Regla: el navegador nunca debe recibir un stack trace, un mensaje de PostgreSQL ni el token.

---

7. Modelo de datos del storefront

7.1 Tipos principales

```
export type PaymentMethod = "CASH" | "DEBIT" | "CREDIT";
export type PaymentStatus =
  | "PENDING"
  | "PROCESSING"
  | "APPROVED"
  | "REJECTED"
  | "REVERSED"
  | "REVERSAL_PENDING"
  | "REVERSAL_FAILED";

export type Fulfillment = "DELIVERY" | "PICKUP";
```

```ts
export type StoreProduct = {
  id: number;
  name: string;
  price: number;
  stock: number;
  categoryId: number | null;
  categoryName: string | null;
  img: string | null;
};

export type StoreCategory = {
  id: number;
  name: string;
  slug: string;
};

export type CartItem = {
  productId: number;
  name: string;
  price: number;
  img: string | null;
  quantity: number;
  stock: number;
};

export type CustomerData = {
  name: string;
  phone: string;
  email?: string;
};

export type DeliveryData = {
  street: string;
  exteriorNumber: string;
  interiorNumber?: string;
  neighborhood: string;
  city: string;
  state: string;
  zip: string;
  references?: string;
};

export type OrderMetadata = {
  version: 1;
```

```
  publicId: string;
  saleId: number;
  sessionId: string;
  idempotencyKey: string;
  createdAt: string;
  fulfillment: Fulfillment;
  customer: CustomerData;
  delivery: DeliveryData | null;
  notes?: string;
  paymentMethod: PaymentMethod;
  paymentStatus: PaymentStatus;
  total: number;
  items: OrderItemSnapshot[];
  lastKnownStatus: string;
  lastSyncedAt: string | null;
};

export type OrderItemSnapshot = {
  productId: number;
  name: string;
  unitPrice: number;
  quantity: number;
  subtotal: number;
  img: string | null;
};

export type OrderView = OrderMetadata & {
  live: {
    confirmed: boolean;
    cancelled: boolean;
    paymentStatus: PaymentStatus;
    saleDate: string;
    total: number;
  } | null;
  uiStatus: OrderUiStatus;
};
```

7.2 Estados de la interfaz

```
export type OrderUiStatus =
  | "RECIBIDO"
  | "EN_PREPARACION"
  | "ENTREGADO"
  | "RECOGIDO"
```

```
  | "CANCELADO"
  | "PAGO_RECHAZADO"
  | "REEMBOLSADO";
```

7.3 Mapeo de estado

Condición del backend	Estado UI	Etiqueta
`cancelled = true`	`CANCELADO`	Cancelado
`confirmed = true` y entrega	`ENTREGADO`	Entregado
`confirmed = true` y recogida	`RECOGIDO`	Recogido
`paymentStatus = REVERSED`	`REEMBOLSADO`	Reembolsado
`paymentStatus = REJECTED`	`PAGO_RECHAZADO`	Pago rechazado
Abierta, menos de 2 minutos	`RECIBIDO`	Recibido
Abierta, 2 minutos o más	`EN_PREPARACION`	En preparación

Importante: `EN_PREPARACION` es una estimación de la UI, no un estado del backend. La pantalla debe decir "Estado estimado" y explicar que el pedido se actualiza cuando el local lo confirma.

7.4 Dinero

- El backend usa `BigDecimal` y lo serializa como número.
- La UI debe mostrar MXN con dos decimales.
- Para sumar, se trabaja en centavos:

```
export function toCents(value: number): number {
  return Math.round(value * 100);
}

export function fromCents(cents: number): number {
  return cents / 100;
}

export function formatMXN(value: number): string {
  return new Intl.NumberFormat("es-MX", {
    style: "currency",
    currency: "MXN",
  }).format(value);
}
```

- El total del carrito es solo para mostrar. El total real lo calcula el backend.
- Si el total del backend difiere del total mostrado, se usa el del backend y se avisa al cliente.

- No se inventan impuestos ni descuentos.
- No se suma envío porque no está implementado.

7.5 Slugs de categoría

- Se generan a partir del nombre:

```
export function slugify(value: string): string {
  return value
    .normalize("NFD")
    .replace(/[\u0300-\u036f]/g, "")
    .toLowerCase()
    .trim()
    .replace(/[^a-z0-9]+/g, "-")
    .replace(/^-+|-+$/g, "");
}
```

- Si dos categorías producen el mismo slug, se agrega el id: `categoria-12`.
- La ruta `/categoria/[slug]` resuelve el nombre real de la categoría y consulta el backend por nombre.
- Si no existe, se muestra `notFound()`.

7.6 Validación

Validaciones mínimas:

Campo	Regla
Nombre	2 a 80 caracteres, no solo espacios
Teléfono	10 dígitos, se normaliza quitando espacios y guiones
Correo	Opcional; si viene, formato válido y máximo 120
Calle	3 a 120 caracteres
Número exterior	1 a 20 caracteres
Número interior	Opcional, máximo 20
Colonia	3 a 100 caracteres
Ciudad	3 a 100 caracteres
Estado	3 a 100 caracteres
Código postal	5 dígitos
Referencias	Opcional, máximo 200
Notas	Opcional, máximo 300
Carrito	1 a 50 líneas
Cantidad	1 a 99 y no mayor al stock

Campo	Regla
Pago	`CASH`, `DEBIT` o `CREDIT`
Entrega	`DELIVERY` o `PICKUP`

La misma validación debe correr en el cliente y en el BFF. La del cliente es UX; la del servidor es la frontera de confianza.

---

8. Flujos

8.1 Catálogo

1. El servidor de Next pide productos y categorías al backend.
2. El BFF autentica con el usuario técnico.
3. El BFF filtra productos inactivos (el backend ya los excluye, pero se valida).
4. El BFF elimina campos internos.
5. El servidor renderiza la página.
6. El cliente hidrata y puede buscar sin recargar.

Reglas:

- Solo se muestran productos con `active = true`.
- Un producto con stock 0 se muestra como "Agotado" y no se puede agregar.
- No se muestra el costo.
- No se muestra el SKU ni el código de barras.
- Si no hay imagen, se usa un placeholder local.
- Si una imagen falla, se muestra el placeholder, no un ícono roto.

8.2 Categorías

1. El BFF obtiene `GET /api/local/categories`.
2. Genera slugs.
3. El menú muestra las categorías con productos.
4. Al entrar a `/categoria/[slug]`, se resuelve el nombre.
5. Se pide `GET /api/local/products?category=<nombre>`.
6. Se muestran los productos.

Si una categoría no tiene productos, se muestra un estado vacío honesto.

8.3 Búsqueda

- La caja de búsqueda tiene debounce de 300 ms.
- Si la consulta está vacía, se muestra el catálogo normal.
- Si tiene texto, se llama a `GET /api/local/products/search?q=...`.
- Se muestran resultados y un botón para limpiar.
- Si no hay resultados, se muestra un estado vacío.
- La búsqueda no distingue mayúsculas/minúsculas (el backend ya lo resuelve).

8.4 Carrito

- El carrito vive en `localStorage` bajo una clave versionada: `tienda.cart.v1`.
- Al cargar, se hidrata en un contexto de React.
- Se ignoran datos corruptos.
- Al agregar un producto, si ya existe, se suma la cantidad.
- La cantidad máxima es el stock conocido.
- Si el stock cambia, el checkout vuelve a validar.
- El carrito se sincroniza entre pestañas con el evento `storage` (opcional, recomendado).
- El carrito no se envía al backend hasta el checkout.

8.5 Checkout

Pasos de la UI:

1. **Entrega:** domicilio o recoger en tienda.
2. **Datos:** nombre, teléfono, correo opcional.
3. **Dirección:** solo si es domicilio.
4. **Pago:** efectivo demo o tarjeta simulada.
5. **Revisión:** productos, cantidades, total, datos y método de pago.
6. **Confirmación:** se envía el pedido.

Reglas:

- No se permite confirmar con carrito vacío.
- No se permite confirmar sin caja abierta.
- No se permite confirmar con stock insuficiente.
- El botón de confirmar se deshabilita mientras se envía.
- El mismo `idempotencyKey` se reutiliza en reintentos.
- Si el backend responde con un pedido ya existente para la misma clave, se trata como éxito.
- Si el pago con tarjeta se rechaza, se mantiene el carrito y se muestra el error.
- Si el pedido se crea pero falla el guardado de metadatos, se muestra el pedido igual y se registra la advertencia. **Nunca se debe decir que falló un pedido que sí se cobró.**

8.6 Secuencia del BFF al crear un pedido

```
1. Recibir POST /api/bff/pedidos
2. Verificar Origin y Content-Type
3. Aplicar rate limit por IP/sesión
4. Validar el cuerpo
5. Verificar que hay caja abierta
6. Pedir el catálogo actual al backend
7. Validar que cada producto existe y está activo
8. Validar stock
9. Calcular el total en centavos
10. Construir SaleRequest
11. Enviar POST /api/local/sales con idempotencyKey
12. Si el backend rechaza, mapear el error y no guardar metadatos
13. Si el backend aprueba, guardar OrderMetadata
14. Si el guardado de metadatos falla, devolver el pedido con advertencia
15. Devolver publicId, saleId, total y estado
```

8.7 Idempotencia

- La clave se genera al abrir el checkout, no al hacer clic en confirmar.
- Se guarda en `sessionStorage` bajo `tienda.checkout.idempotencyKey`.
- Se reutiliza en cada reintento del mismo checkout.
- Se limpia cuando el pedido se confirma con éxito.
- Si el usuario recarga la página, la clave se recupera de `sessionStorage`.
- Si el usuario abre un checkout nuevo, se genera una clave nueva.
- La clave tiene formato UUID y máximo 64 caracteres.

8.8 Seguimiento del pedido

- Al crear el pedido, se guarda `OrderMetadata` y se redirige a `/pedidos/[publicId]`.
- La página lee la metadata y consulta `GET /api/local/sales/{saleId}`.
- El estado se actualiza cada 10 segundos mientras la página está visible.
- El polling se pausa cuando la pestaña está oculta.
- El polling se detiene cuando el pedido está cancelado, entregado o recogido.
- Si el backend no responde, se muestra el último estado conocido y un aviso.
- La lista `/pedidos` usa la metadata guardada; no hace una llamada por pedido.
- El detalle es el único que consulta el estado en vivo.

8.9 Volver a pedir

- Desde el detalle, el botón “Volver a pedir” toma los productos del pedido.

- Consulta el catálogo actual.
- Agrega solo productos activos y con stock.
- Si un producto ya no existe, se omite y se avisa.
- Si el precio cambió, se usa el precio actual y se avisa.
- Si no queda ningún producto, se muestra un estado vacío.

---

9. Endpoints del BFF

9.1 `GET /api/bff/catalogo`

Query params:

- `categoria` (opcional): nombre de categoría.
- `q` (opcional): búsqueda.
- `orden` (opcional): `nombre`, `precio-asc`, `precio-desc`.

Respuesta:

```
{
  "products": [
    {
      "id": 1,
      "name": "Arroz 1 kg",
      "price": 28.5,
      "stock": 12,
      "categoryId": 2,
      "categoryName": "Despensa",
      "img": "http://localhost:8081/api/uploads/arroz.jpg"
    }
  ],
  "categories": [
    { "id": 2, "name": "Despensa", "slug": "despensa" }
  ],
  "updatedAt": "2026-10-22T14:30:00.000Z"
}
```

Errores:

- `503 ERROR_BACKEND` si el backend no responde.
- `503 ERROR_BACKEND` si el login técnico falla.

9.2 `GET /api/bff/productos/[id]`

Respuesta:

```
{
  "product": {
    "id": 1,
    "name": "Arroz 1 kg",
    "price": 28.5,
    "stock": 12,
    "categoryId": 2,
    "categoryName": "Despensa",
    "img": "http://localhost:8081/api/uploads/arroz.jpg"
  },
  "related": []
}
```

Nota: el backend no tiene `GET /products/{id}`. El BFF descarga el catálogo y filtra. Si el producto no existe o está inactivo, devuelve `404`.

9.3 `GET /api/bff/categorias`

```
{
  "categories": [
    { "id": 2, "name": "Despensa", "slug": "despensa" }
  ]
}
```

9.4 `GET /api/bff/estado-tienda`

```
{
  "open": true,
  "cashNumber": "CAJA 1",
  "checkedAt": "2026-10-22T14:30:00.000Z"
}
```

Si no hay caja:

```
{
  "open": false,
  "cashNumber": null,
```

```
  "checkedAt": "2026-10-22T14:30:00.000Z"
}
```

9.5 `POST /api/bff/pedidos`

Request:

```
{
  "fulfillment": "DELIVERY",
  "customer": {
    "name": "Ana Pérez",
    "phone": "5512345678",
    "email": "ana@example.com"
  },
  "delivery": {
    "street": "Av. Reforma",
    "exteriorNumber": "123",
    "interiorNumber": "4B",
    "neighborhood": "Centro",
    "city": "Ciudad de México",
    "state": "CDMX",
    "zip": "06000",
    "references": "Portón verde"
  },
  "notes": "Tocar el timbre",
  "paymentMethod": "CASH",
  "items": [
    { "productId": 1, "quantity": 2 }
  ],
  "idempotencyKey": "0f8fad5b-d9cb-469f-a165-70867728950e"
}
```

Respuesta exitosa:

```
{
  "publicId": "PED-20261022-A1B2C3",
  "saleId": 123,
  "total": 250.0,
  "paymentMethod": "CASH",
  "paymentStatus": "APPROVED",
  "metadataSaved": true
}
```

Errores:

- `400 DATOS_INVALIDOS`
- `409 PRODUCTO_NO_DISPONIBLE`
- `409 STOCK_INSUFICIENTE`
- `402 PAGO_RECHAZADO`
- `503 TIENDA_CERRADA`
- `503 ERROR_BACKEND`
- `429 RATE_LIMITED`

9.6 `GET /api/bff/pedidos`

Lista los pedidos de la sesión anónima.

```
{
  "orders": [
    {
      "publicId": "PED-20261022-A1B2C3",
      "saleId": 123,
      "createdAt": "2026-10-22T14:30:00",
      "total": 250.0,
      "fulfillment": "DELIVERY",
      "uiStatus": "RECIBIDO",
      "itemCount": 2
    }
  ]
}
```

9.7 `GET /api/bff/pedidos/[publicId]`

```
{
  "order": {
    "publicId": "PED-20261022-A1B2C3",
    "saleId": 123,
    "createdAt": "2026-10-22T14:30:00",
    "fulfillment": "DELIVERY",
    "customer": {
      "name": "Ana Pérez",
      "phone": "5512345678"
    },
    "delivery": {
      "street": "Av. Reforma",
      "exteriorNumber": "123",
      "neighborhood": "Centro",
      "city": "Ciudad de México",
```

```
      "state": "CDMX",
      "zip": "06000"
    },
    "items": [],
    "total": 250.0,
    "paymentMethod": "CASH",
    "paymentStatus": "APPROVED",
    "uiStatus": "EN_PREPARACION",
    "lastSyncedAt": "2026-10-22T14:35:00.000Z"
  }
}
```

Seguridad: si el `publicId` existe pero pertenece a otra sesión, devolver `404`, no `403`. No se revela que el pedido existe.

9.8 Formato de error

```
{
  "error": {
    "code": "STOCK_INSUFICIENTE",
    "message": "Un producto ya no tiene existencias suficientes.",
    "details": [
      { "productId": 1, "available": 1, "requested": 2 }
    ]
  }
}
```

`details` es opcional. Nunca debe incluir datos internos del backend.

---

10. Almacén temporal de pedidos

10.1 Por qué existe

El backend guarda la venta, pero no guarda:

- nombre del cliente;
- teléfono;
- correo;
- dirección;
- tipo de entrega;

- notas;
- número público de pedido.

Esa información es necesaria para la experiencia de tienda. Se guarda aparte, sin tocar el esquema del backend.

10.2 Interfaz

```
export interface OrderStore {
  create(order: OrderMetadata): Promise<void>;
  findByPublicId(publicId: string): Promise<OrderMetadata | null>;
  findBySaleId(saleId: number): Promise<OrderMetadata | null>;
  listBySession(sessionId: string): Promise<OrderMetadata[]>;
  update(order: OrderMetadata): Promise<void>;
}
```

10.3 Implementación para la demo local

- Archivo: `.data/pedidos.json` o `ORDER_STORE_PATH`.
- Estructura:

```
{
  "version": 1,
  "orders": []
}
```

- Se crea el directorio si no existe.
- Se lee el archivo completo en cada operación (el volumen es pequeño).
- Se escribe con archivo temporal y `rename` para evitar corrupción.
- Se usa un mutex en memoria para evitar escrituras simultáneas.
- Si el archivo está corrupto, se hace una copia `.bak` y se empieza de cero, registrando el error.
- `.data/` debe estar en `.gitignore`.

10.4 Limitaciones

- No es seguro para múltiples procesos o instancias.
- No es durable en Vercel, Netlify Functions ni contenedores efímeros.
- No es una base de datos.
- Guarda datos personales en texto plano.
- No tiene cifrado en reposo.
- No tiene retención automática.

- No sirve para producción.

Regla: este almacén es para la demo local y para validar el flujo. Si se despliega, se debe reemplazar por una base de datos o por el futuro backend de pedidos.

10.5 Sesión anónima

- Cookie: `tienda_sid`.
- Valor: UUID aleatorio.
- `httpOnly: true`.
- `sameSite: "lax"`.
- `secure: true` en producción.
- `path: "/"`.
- `maxAge`: 30 días.
- Se crea en el primer `POST /api/bff/pedidos` si no existe.
- El BFF lee la cookie con `await cookies()`.
- Si no hay cookie en `GET /pedidos`, se muestra una lista vacía.
- Si el usuario borra la cookie, pierde el historial.
- No se usa para autenticar; solo para separar pedidos de navegadores.

Limitación: no es autenticación real. Un usuario con la cookie puede ver esos pedidos. Para la demo es suficiente; para producción se necesita login de cliente.

10.6 Número público de pedido

Formato:

```
PED-YYYYMMDD-XXXXXX
```

- `YYYYMMDD` es la fecha local.
- `XXXXXX` son 6 caracteres aleatorios en mayúsculas.
- No es secuencial.
- No se usa como secreto; la protección es la cookie de sesión.
- Se guarda junto con `saleId`.

10.7 Idempotencia del almacén

- Antes de crear, buscar por `idempotencyKey`.
- Si ya existe, devolver el pedido existente.
- Si la venta ya existe en el backend pero la metadata no, buscar por `saleId`.

- Si la metadata falla, no se pierde la venta: se puede reconstruir con `saleId` y los datos del request en memoria, pero no se debe prometer persistencia.

---

11. Rutas y pantallas

11.1 Mapa de rutas

Ruta	Tipo	Descripción
`/`	Server Component	Home con categorías y productos
`/categoria/[slug]`	Server Component	Productos de una categoría
`/platillo/[id]`	Server Component	Detalle de producto
`/buscar`	Client/Server	Resultados de búsqueda
`/carrito`	Client Component	Carrito completo
`/checkout`	Client Component	Formulario y pago
`/pedidos`	Server Component	Pedidos de la sesión
`/pedidos/[publicId]`	Server + Client	Detalle y polling
`/login`	Stub	"Cuenta de cliente próximamente"
`/registro`	Stub	"Cuenta de cliente próximamente"
`/local/pedidos`	Opcional	Panel interno de demo

11.2 Rutas existentes que se deben ajustar

- `app/categoria/page.tsx` → mover a `app/categoria/[slug]/page.tsx` o convertir en índice de categorías.
- `app/platillo/page.tsx` → mover a `app/platillo/[id]/page.tsx`.
- `app/pedidos/pageId.tsx` → eliminar; crear `app/pedidos/[publicId]/page.tsx`.
- `app/login/page.tsx` y `app/registro/page.tsx` → dejar como stubs informativos, sin formularios falsos.
- `app/checkout/page.tsx` → implementar.
- `app/carrito/page.tsx` → implementar.
- `app/page.tsx` → implementar.

11.3 Layout

Header:

- Logo o nombre de la tienda.
- Navegación: Inicio, Categorías, Pedidos.
- Buscador.
- Botón de carrito con contador.

- En móvil: menú compacto y barra inferior.

Footer:

- Nombre de la tienda.
- Aviso de demo.
- Enlaces a Pedidos y Ayuda.
- Horario: no inventar; usar un texto configurable o “Consulta horarios en la tienda”.

Barra inferior móvil:

- Inicio.
- Buscar.
- Carrito.
- Pedidos.

11.4 Home

Secciones:

1. Hero con mensaje claro: “Tu despensa, a domicilio”.
2. Aviso de demo: “Pagos simulados. No uses tarjetas reales.”
3. Categorías.
4. Productos disponibles.
5. Cómo funciona: elige, confirma, recibe.
6. Aviso de que el costo de envío no está implementado.

No incluir:

- Testimonios inventados.
- “Ofertas” si no hay descuentos.
- “Envío gratis” si no es una regla del negocio.
- Contadores falsos.
- Reseñas falsas.

11.5 Categoría

- Título de la categoría.
- Cantidad de productos.
- Orden por nombre o precio.
- Grid de productos.
- Estado vacío.
- Enlace al catálogo completo.

11.6 Producto

- Imagen grande con fallback.
- Nombre.
- Categoría.
- Precio.
- Stock: “Disponible”, “Últimas unidades” o “Agotado”.
- Selector de cantidad.
- Botón “Agregar al carrito”.
- Aviso de pago simulado si aplica.
- Productos relacionados de la misma categoría.
- No inventar descripción, ingredientes, peso ni caducidad si el backend no los tiene.

11.7 Carrito

- Lista de líneas con imagen, nombre, precio unitario, cantidad y subtotal.
- Botones + y -.
- Eliminar.
- Vaciar carrito con confirmación.
- Resumen: subtotal, envío demo, total.
- Botón “Continuar al checkout”.
- Estado vacío con enlace al catálogo.
- Aviso si un producto quedó sin stock.

11.8 Checkout

- Formulario en dos columnas en escritorio, una en móvil.
- Resumen del pedido fijo o al final.
- Validación inline.
- Botón de confirmar con estado de carga.
- Deshabilitado si falta algo.
- Aviso de que el pago es simulado.
- Aviso de que el pedido se registra en el POS.
- Enlace a la política de demo: no usar datos reales de tarjeta.

11.9 Pedidos

- Lista de pedidos de la sesión.
- Tarjeta con número, fecha, total, estado y tipo de entrega.
- Estado vacío.

- Enlace al detalle.
- No mostrar pedidos de otras sesiones.

11.10 Detalle de pedido

- Número público.
- Fecha.
- Estado estimado con timeline.
- Productos.
- Dirección o instrucción de recogida.
- Datos de contacto.
- Pago.
- Total.
- Botón “Actualizar”.
- Botón “Volver a pedir”.
- Aviso de que el estado se actualiza cuando el local confirma la venta.
- Si está cancelado, aviso claro.
- Si el backend no responde, último estado conocido.

11.11 Panel interno opcional

Ruta: `/local/pedidos`.

- Solo para demo.
- Protegido por `TIENDA_ADMIN_TOKEN`.
- Login simple con token; cookie `tienda_admin`.
- Lista de pedidos con metadata y estado del backend.
- Permite confirmar la venta (`PATCH /sales/{id}/confirm`) si el usuario técnico tiene `CONFIRMAR_VENTAS`.
- No permite anular; para anular se usa Angular.
- No se enlaza desde la tienda pública.
- No se implementa en la primera fase; es opcional.

---

12. Diseño visual

12.1 Dirección

- Cercano, limpio y confiable.
- Estilo despensa/mercado, no restaurante de lujo.

- Fondo cálido claro.
- Verde como color principal.
- Ámbar para avisos.
- Rojo solo para errores.
- Bordes redondeados y sombras suaves.
- Tipografía Geist, ya instalada.

12.2 Tokens

```
@theme {
  --color-bg: #fbf8f1;
  --color-surface: #ffffff;
  --color-ink: #1f2a24;
  --color-muted: #66736b;
  --color-primary: #1f7a4d;
  --color-primary-strong: #145c38;
  --color-accent: #e9a23b;
  --color-danger: #c2413b;
  --color-border: #e5e0d5;
  --radius-card: 1rem;
  --shadow-card: 0 8px 24px rgb(31 42 36 / 0.08);
}
```

12.3 Componentes base

- `Button`
- `IconButton`
- `Input`
- `Textarea`
- `Select`
- `Badge`
- `Card`
- `Dialog`
- `Drawer`
- `QuantityStepper`
- `Price`
- `EmptyState`
- `Skeleton`
- `Toast`
- `StatusBadge`

- `OrderTimeline`

No se agrega una librería de componentes en la primera fase. Tailwind es suficiente. Los iconos se hacen con SVG inline para no sumar dependencias.

12.4 Accesibilidad

- HTML semántico.
- Un solo `h1` por página.
- Etiquetas visibles en los campos.
- `aria-live` para errores y actualizaciones de estado.
- Foco visible.
- Navegación por teclado.
- Contraste mínimo AA.
- `prefers-reduced-motion`.
- Imágenes con `alt`.
- Botones con texto o `aria-label`.
- Diálogos con foco atrapado y cierre con Escape.
- No depender solo del color para indicar estado.

12.5 Responsive

- Mobile first.
- Grid de 1, 2, 3 o 4 columnas según ancho.
- Header compacto en móvil.
- Barra inferior de navegación.
- Checkout de una columna en móvil.
- Tablas convertidas en tarjetas.

---

13. SEO y metadata

- `metadata` por página:
- Home: nombre de la tienda + “Despensa a domicilio”.
- Categoría: nombre de la categoría.
- Producto: nombre + precio + categoría.
- `generateMetadata` para rutas dinámicas.
- Open Graph básico.
- `robots.ts` y `sitemap.ts`.

- JSON-LD:
- `Product` con `Offer` y `priceCurrency: "MXN"`.
- `BreadcrumbList`.
- No indexar `/checkout`, `/carrito`, `/pedidos` ni `/local`.
- No incluir datos personales en metadata.
- No generar metadata estática con precios que cambian; usar datos del servidor.

---

14. Seguridad

14.1 Reglas

- El JWT del backend nunca se envía al navegador.
- Los secretos solo viven en variables server-only.
- No se usan datos de tarjeta reales.
- No se registran números de tarjeta, PIN ni tokens.
- No se registran direcciones completas en logs de producción.
- El BFF valida `Origin` en mutaciones.
- El BFF valida `Content-Type: application/json`.
- El BFF limita el tamaño del cuerpo.
- El BFF aplica rate limit básico en memoria.
- El BFF no devuelve errores internos.
- El BFF no confía en precios ni totales del cliente.
- El BFF no confía en `stock` del cliente.
- El BFF no acepta `saleId` del cliente.
- El BFF no acepta `userId` del cliente.
- El BFF no acepta `cost`.
- El BFF no permite modificar la venta después de crearla.
- El BFF no expone el panel interno en la navegación pública.
- La cookie de sesión es `httpOnly` y `SameSite=Lax`.
- La cookie de admin es `httpOnly` y separada.
- Los formularios muestran un aviso de demo.

14.2 CSRF

- La tienda no tiene login de cliente, pero sí cookie de sesión.
- Las mutaciones verifican `Origin` contra `NEXT_PUBLIC_SITE_URL`.

- Si el origen no coincide, `403`.
- No se aceptan mutaciones desde formularios de otros dominios.
- Para producción se recomienda un token CSRF de doble envío.

14.3 Rate limit

- Implementación en memoria por IP y por sesión.
- Límite sugerido:
- `POST /api/bff/pedidos`: 5 por minuto por IP.
- `GET /api/bff/catalogo`: 60 por minuto por IP.
- Login del panel: 5 por minuto por IP.
- No es distribuido; suficiente para demo.
- Si se despliega en varias instancias, usar un almacén compartido.

14.4 Datos personales

- Se guardan solo los necesarios.
- No se guarda tarjeta ni PIN.
- Se guarda `lastFourDigits` solo si el backend lo devuelve.
- El archivo de pedidos está en `.data/`, fuera de git.
- Se documenta cómo borrarlo.
- No se envían datos personales a servicios externos.
- No se usan analytics en la primera fase.

14.5 Cabeceras

En `next.config.ts` o `proxy.ts`:

- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: strict-origin-when-cross-origin`
- `X-Frame-Options: DENY`
- `Permissions-Policy: camera=(), microphone=(), geolocation=()`
- `Content-Security-Policy`: evaluar en una fase posterior; Next necesita ajustes por scripts inline.

---

15. Pruebas

15.1 Pruebas unitarias

Herramienta recomendada: Vitest.

Casos:

- `toCents`, `fromCents`, `formatMXN`.
- `slugify`.
- Validación de nombre, teléfono, correo, dirección, CP.
- Validación de carrito y cantidades.
- Mapeo de `ProductDto` a `StoreProduct`.
- Mapeo de estado de pedido.
- Generación de `publicId`.
- Idempotencia del `OrderStore` (misma clave devuelve el mismo pedido).
- Parseo de errores del backend.
- Sanitización de producto (no expone `cost`).

15.2 Pruebas de integración del BFF

Con Vitest y `fetch` mockeado:

- Login exitoso y cache de token.
- Re-login ante 401.
- Error si el login falla.
- Catálogo sin campos internos.
- Producto inexistente → 404.
- Sin caja → 503.
- Stock insuficiente → 409.
- Tarjeta rechazada → 402.
- Pedido exitoso → metadata guardada.
- Doble pedido con la misma clave → un solo pedido.
- Pedido de otra sesión → 404.
- Error de backend → 503 con mensaje genérico.

15.3 Pruebas E2E

Herramienta recomendada: Playwright.

Escenarios:

1. Home carga productos.
2. Categoría filtra productos.
3. Búsqueda encuentra un producto.
4. Agregar al carrito y recargar conserva el carrito.
5. Checkout en efectivo crea venta y muestra pedido.

6. La venta aparece en Angular.
7. El stock baja en Angular.
8. Doble clic en confirmar crea una sola venta.
9. Tarjeta rechazada muestra error y no crea venta.
10. Sin caja abierta muestra tienda cerrada.
11. Confirmar en Angular actualiza el estado en Next.
12. Anular en Angular muestra cancelado en Next.
13. Otra sesión no ve el pedido.
14. Volver a pedir agrega productos disponibles.
15. Responsive en 375 px, 768 px y 1280 px.

15.4 Pruebas manuales mínimas

Si no se agrega Playwright:

- Ejecutar el flujo completo con backend y Angular.
- Verificar en la base o en Angular que la venta existe.
- Verificar el stock antes y después.
- Verificar doble clic.
- Verificar tarjeta rechazada.
- Verificar caja cerrada.
- Verificar recarga de página.
- Verificar pedidos en otra ventana.
- Verificar consola sin errores.
- Verificar que el token no aparece en el navegador.

15.5 Comandos de verificación

```
npm run lint
npx tsc --noEmit
npm run build
```

Si se agregan tests:

```
npm run test
npm run test:e2e
```

---

16. Fases de implementación

Fase 0 — Preparación

Objetivo: dejar el entorno listo y comprobar que el backend responde.

Tareas:

1. Arrancar PostgreSQL.
2. Arrancar backend en `8081`.
3. Arrancar Angular en `4200`.
4. Verificar productos, categorías e imágenes.
5. Crear rol `TIENDA`.
6. Crear usuario técnico.
7. Abrir caja.
8. Crear `.env.local` con las variables.
9. Agregar `.data/` a `.gitignore`.
10. Leer la documentación local de Next 16.
11. Probar login y una venta de prueba.
12. Anular la venta de prueba.

Verificación:

- `POST /api/local/auth/login` responde 200 con token.
- El usuario técnico puede `GET /api/local/products`.
- El usuario técnico puede `POST /api/local/sales` con una venta de prueba.
- `GET /api/local/cash/active` responde 200.
- La imagen de un producto carga en el navegador.

Fase 1 — Fundación

Objetivo: layout, configuración, cliente del backend y salud.

Tareas:

1. Configurar `next.config.ts` con `images.remotePatterns`.
2. Actualizar `app/layout.tsx` con metadata, header y footer.
3. Definir tokens en `app/globals.css`.
4. Crear `lib/server/env.ts`.
5. Crear `lib/server/backend/client.ts`.
6. Crear `lib/server/backend/auth.ts`.
7. Crear `lib/server/backend/errors.ts`.

8. Crear `lib/server/backend/products.ts`.
9. Crear `lib/server/backend/categories.ts`.
10. Crear `lib/server/backend/sales.ts`.
11. Crear `lib/server/backend/cash.ts`.
12. Crear `app/api/bff/health/route.ts`.
13. Crear componentes base.
14. Crear `app/not-found.tsx` y `app/error.tsx`.

Verificación:

- `/api/bff/health` responde con backend, auth y caja.
- El token no aparece en el HTML ni en el JavaScript del navegador.
- `npm run build` compila.
- La home carga sin errores.

Fase 2 — Catálogo

Objetivo: mostrar productos reales.

Tareas:

1. Crear tipos y mappers.
2. Crear `GET /api/bff/catalogo`.
3. Crear `GET /api/bff/categorias`.
4. Crear `GET /api/bff/productos/[id]`.
5. Implementar home.
6. Implementar `/categoria/[slug]`.
7. Implementar `/platillo/[id]`.
8. Implementar `/buscar`.
9. Implementar imagen con fallback.
10. Implementar estados de carga y vacío.

Verificación:

- El catálogo coincide con los productos activos del POS.
- Un producto dado de baja no aparece.
- Un producto con stock 0 aparece como agotado.
- La categoría filtra correctamente.
- La búsqueda funciona.
- No se muestra `cost` ni `hasHistory`.
- Las imágenes cargan.

Fase 3 — Carrito

Objetivo: armar un carrito persistente.

Tareas:

1. Crear `lib/shared/cart/types.ts`.
2. Crear `lib/shared/cart/storage.ts`.
3. Crear `CartProvider`.
4. Crear `CartButton`.
5. Crear `CartDrawer`.
6. Crear `/carrito`.
7. Implementar sumar, restar, eliminar y vaciar.
8. Limitar cantidad al stock.
9. Sincronizar entre pestañas.

Verificación:

- El carrito sobrevive a recargar.
- El total se calcula bien.
- No se puede agregar más que el stock.
- El carrito vacío muestra estado vacío.
- No hay errores de hidratación.

Fase 4 — Checkout y creación de pedido

Objetivo: crear una venta real desde la tienda.

Tareas:

1. Crear `/checkout`.
2. Crear formulario de entrega y datos.
3. Crear selección de pago.
4. Crear resumen.
5. Crear `POST /api/bff/pedidos`.
6. Implementar validación de servidor.
7. Implementar verificación de caja.
8. Implementar revalidación de catálogo y stock.
9. Implementar cálculo de total.
10. Implementar idempotencia.
11. Implementar mapeo de errores.
12. Implementar pantalla de éxito.

Verificación:

- Un pedido en efectivo crea una venta.
- El stock baja.
- La venta aparece en Angular.
- Un doble clic no crea dos ventas.
- Un reintento con la misma clave no crea dos ventas.
- Una tarjeta rechazada no crea venta.
- Sin caja se muestra un error claro.
- Un precio manipulado en el navegador no afecta el total del backend.
- Un producto sin stock no se puede pedir.

Fase 5 — Pedidos y seguimiento

Objetivo: que el cliente vea su pedido.

Tareas:

1. Crear `OrderStore`.
2. Crear `lib/server/orders/file-store.ts`.
3. Crear cookie de sesión.
4. Guardar metadata al crear el pedido.
5. Crear `GET /api/bff/pedidos`.
6. Crear `GET /api/bff/pedidos/[publicId]`.
7. Crear `/pedidos`.
8. Crear `/pedidos/[publicId]`.
9. Implementar polling.
10. Implementar mapeo de estado.
11. Implementar "Volver a pedir".

Verificación:

- El pedido aparece en `/pedidos`.
- El pedido sobrevive a recargar.
- Otra sesión no ve el pedido.
- El estado cambia al confirmar en Angular.
- El estado cambia al anular en Angular.
- El polling se detiene en estados finales.
- La página funciona si el backend se cae (muestra último estado).

Fase 6 — Panel interno opcional

Objetivo: facilitar la demo sin cambiar de ventana.

Tareas:

1. Crear `/local/pedidos`.
2. Proteger con `TIENDA_ADMIN_TOKEN`.
3. Listar pedidos.
4. Mostrar estado del backend.
5. Confirmar venta si hay permiso.
6. Enlazar a Angular para anular.

Verificación:

- Sin token no se puede entrar.
- Con token se ven los pedidos.
- Confirmar actualiza el estado en la tienda.
- No se expone en la navegación pública.

Fase 7 — Calidad

Objetivo: dejar la tienda presentable y estable.

Tareas:

1. Pruebas unitarias.
2. Pruebas de integración.
3. Pruebas E2E o checklist manual.
4. SEO.
5. Accesibilidad.
6. Responsive.
7. Manejo de errores.
8. Estados de carga.
9. Revisión de seguridad.
10. Documentación de limitaciones.

Verificación:

- `npm run lint` sin errores.
- `npx tsc --noEmit` sin errores.
- `npm run build` exitoso.
- Flujo completo probado.
- Lighthouse aceptable en home y producto.

- No hay secretos en el bundle.

Fase 8 — Demo y despliegue

Objetivo: preparar una demo reproducible.

Tareas:

1. Documentar cómo arrancar backend, Angular y Next.
2. Documentar variables de entorno.
3. Documentar creación de rol y usuario.
4. Documentar apertura de caja.
5. Documentar tarjetas de prueba.
6. Documentar limitaciones.
7. Si se despliega, reemplazar el archivo de pedidos por un almacén durable.
8. Configurar `NEXT_PUBLIC_SITE_URL`.
9. Configurar el backend público.
10. Verificar CORS solo si se llama directo; con BFF no hace falta.

Verificación:

- Un compañero puede repetir la demo desde cero.
- No se necesitan secretos en el repositorio.
- El flujo funciona en el entorno acordado.

---

17. Archivos previstos

```
app/
  api/
    bff/
      catalogo/route.ts
      categorias/route.ts
      estado-tienda/route.ts
      health/route.ts
      pedidos/route.ts
      pedidos/[publicId]/route.ts
      productos/[id]/route.ts
  carrito/page.tsx
  categoria/[slug]/page.tsx
  checkout/page.tsx
  pedidos/page.tsx
```

```
  pedidos/[publicId]/page.tsx
  platillo/[id]/page.tsx
  buscar/page.tsx
  local/pedidos/page.tsx
  error.tsx
  not-found.tsx
  layout.tsx
  page.tsx
  globals.css

components/
  layout/Header.tsx
  layout/Footer.tsx
  layout/MobileNav.tsx
  catalog/ProductCard.tsx
  catalog/ProductGrid.tsx
  catalog/CategoryNav.tsx
  catalog/ProductGallery.tsx
  cart/CartButton.tsx
  cart/CartDrawer.tsx
  cart/CartLine.tsx
  cart/CartSummary.tsx
  checkout/CheckoutForm.tsx
  checkout/DeliveryFields.tsx
  checkout/PaymentFields.tsx
  checkout/OrderReview.tsx
  orders/OrderCard.tsx
  orders/OrderTimeline.tsx
  orders/OrderStatusBadge.tsx
  ui/Button.tsx
  ui/Input.tsx
  ui/Textarea.tsx
  ui/Select.tsx
  ui/Badge.tsx
  ui/Card.tsx
  ui/Dialog.tsx
  ui/Drawer.tsx
  ui/QuantityStepper.tsx
  ui/EmptyState.tsx
  ui/Skeleton.tsx
  ui/Toast.tsx

lib/
  server/
    env.ts
```

```
    backend/
      client.ts
      auth.ts
      errors.ts
      products.ts
      categories.ts
      sales.ts
      cash.ts
      mappers.ts
    orders/
      store.ts
      file-store.ts
      public-id.ts
    session.ts
    rate-limit.ts
  shared/
    types.ts
    money.ts
    validation.ts
    slug.ts
    order-status.ts
    cart/
      types.ts
      storage.ts
      context.tsx

types/
  api.ts
  store.ts

proxy.ts                # opcional, solo si se necesita lógica por request
next.config.ts          # imágenes y cabeceras
.env.local              # gitignored
.data/                  # gitignored
```

---

18. Tarjetas de prueba

Usar solo en la demo. Nunca datos reales.

Tarjeta	Marca	Tipo	PIN	Resultado esperado
`4111111111111111`	VISA	Débito	`1234`	Aprobada
`4000000000000002`	VISA	Débito	`1234`	Rechazada por fondos insuficientes

Tarjeta	Marca	Tipo	PIN	Resultado esperado
`5500000000000003`	Mastercard	Crédito	`1234`	Rechazada por tarjeta bloqueada
`3400000000000004`	American Express	Crédito	`0000`	Rechazada por PIN incorrecto

Reglas de UI:

- Mostrar un selector de "Tarjeta de prueba".
- Mostrar marca, tipo y último resultado esperado.
- No pedir nombre del titular.
- No pedir CVV.
- No pedir fecha de expiración.
- No guardar el número.
- No registrar el PIN.
- No usar la palabra "pago seguro" si es simulado.
- Mostrar "Pago simulado" en todo momento.

---

19. Riesgos y mitigaciones

Riesgo	Probabilidad	Impacto	Mitigación
El backend no tiene caja abierta	Alta	Alto	Verificar y mostrar tienda cerrada
El stock cambia entre el carrito y el checkout	Alta	Medio	Revalidar en servidor y mostrar error claro
Dos pedidos simultáneos sobrevenden	Media	Alto	Idempotencia + revalidación; documentar que no es atómico
El token técnico expira	Media	Medio	Cache y re-login automático
El archivo de pedidos se corrompe	Baja	Medio	Escritura atómica y copia `.bak`
Se pierde el archivo de pedidos	Media	Medio	Documentar; para demo se puede reconstruir desde la venta
El cliente borra la cookie	Media	Bajo	Avisar que el historial es local
Se expone `cost`	Baja	Alto	Mapper estricto y test
Se expone el token	Baja	Muy alto	Variables server-only y revisión del bundle
Se usan tarjetas reales	Media	Muy alto	Avisos, tarjetas de prueba y campos limitados
El backend cambia el contrato	Media	Alto	Tipos y tests de contrato
Se despliega el archivo local en serverless	Alta	Alto	No desplegar así; usar almacén durable
Next 16 cambia APIs	Baja	Medio	Leer docs locales y usar `proxy.ts` / `params` async
Imágenes no cargan	Media	Bajo	`remotePatterns` y fallback
El cliente intenta manipular precios	Media	Medio	El servidor recalcula todo

Riesgo	Probabilidad	Impacto	Mitigación
Se muestran estados falsos	Media	Alto	Mapeo honesto y etiqueta “Estado estimado”
El panel interno queda público	Baja	Alto	Token, cookie separada y no enlazarlo
Se guardan datos personales sin control	Media	Medio	`.data` fuera de git, retención manual, no logs

---

20. Limitaciones que se deben decir al usuario

Estas limitaciones no son defectos del plan; son consecuencias de no tocar el backend.

1. **No hay cuenta de cliente.** El historial depende de la cookie del navegador.
2. **No hay pedidos en el backend.** La dirección y el teléfono viven en un archivo temporal.
3. **No hay estados de cocina.** “En preparación” es una estimación.
4. **No hay “en camino”.** No se debe mostrar.
5. **No hay costo de envío.** El total es solo de productos.
6. **No hay pago real.** Todo pago es simulado.
7. **No hay cancelación en línea.** Cancelar se hace desde el POS.
8. **No hay notificaciones.** El cliente debe consultar la página.
9. **No hay login de cliente.** `/login` y `/registro` quedan informativos.
10. **Las ventas online quedan a nombre del usuario técnico.** No se distingue el cliente en el POS.
11. **El stock no es atómico.** Dos pedidos simultáneos podrían sobrevender.
12. **El archivo de pedidos no es durable.** No sirve para producción.
13. **No hay multi-sucursal.** Una tienda, una caja.
14. **No hay impuestos desglosados.** Se muestra el total del backend.
15. **No hay facturación.** No se ofrece factura.
16. **No hay devoluciones en línea.** Se resuelven en tienda.
17. **No hay analítica.** No se instala tracking en la primera fase.
18. **No hay búsqueda avanzada.** Solo nombre, como el backend.
19. **No hay filtros por precio, marca ni disponibilidad.** Se pueden agregar después.
20. **No hay reseñas ni calificaciones.** No se inventan.

---

21. Migración futura al backend de tienda

Cuando exista `/api/tienda/**` en el backend, este frontend debe poder cambiar sin rehacer la UI.

21.1 Estrategia

- Mantener el BFF como fachada.
- Crear un adaptador `lib/server/backend/local.ts` y otro `lib/server/backend/tienda.ts`.
- Seleccionar con `BACKEND_MODE=local|tienda`.
- La UI sigue hablando con `/api/bff/*`.
- Los tipos compartidos se mantienen.
- Los mapeos cambian por adaptador.
- El almacén temporal se reemplaza por la API de pedidos.

21.2 Qué debe existir en el backend futuro

- Catálogo público de solo lectura.
- Autenticación de cliente.
- Entidad `Pedido` con estados.
- Direcciones.
- Tipo de entrega.
- Costo de envío.
- Pago real.
- Notificaciones.
- Cancelación controlada.
- Historial por cliente.
- Roles de cliente.

21.3 Qué se conserva

- Diseño.
- Rutas.
- Componentes.
- Carrito.
- Validaciones.
- BFF.
- Pruebas.
- Estados de UI.

21.4 Qué se descarta

- Usuario técnico.
- Archivo `.data/pedidos.json`.
- Cookie anónima como única identidad.

- Mapeo de estados derivados.
- Tarjetas de prueba.

---

22. Decisiones abiertas

Estas decisiones no bloquean el plan, pero conviene resolverlas antes de la Fase 2.

1. **Nombre y marca de la tienda.** ¿Se usa el nombre actual o uno nuevo?
2. **Logo y paleta.** ¿Verde/crema o se prefiere otro estilo?
3. **Método de pago por defecto.** ¿Efectivo demo o tarjeta simulada?
4. **Panel interno.** ¿Se implementa `/local/pedidos` o se usa Angular?
5. **Horario de atención.** ¿Se muestra un texto fijo o se omite?
6. **Costo de envío.** ¿Se muestra "sin costo" o "por confirmar"?
7. **Datos de contacto del negocio.** ¿WhatsApp, teléfono, correo?
8. **Tests.** ¿Se agrega Vitest/Playwright ahora o después?
9. **Dependencias.** ¿Se permite `lucide-react`, `zod` o `server-only`?
10. **Despliegue.** ¿Solo demo local o también Vercel?

---

23. Checklist de aceptación

Catálogo

- ☐ La home muestra productos activos.
- ☐ Las categorías se ven y filtran.
- ☐ La búsqueda funciona.
- ☐ El detalle de producto carga.
- ☐ Los productos agotados se marcan.
- ☐ No se muestra el costo.
- ☐ Las imágenes tienen fallback.

Carrito

- ☐ Agregar funciona.
- ☐ Sumar y restar funciona.
- ☐ Eliminar funciona.
- ☐ El carrito sobrevive a recargar.

- ☐ El total es correcto.
- ☐ No se supera el stock.

Checkout

- ☐ Se elige entrega o recogida.
- ☐ Se validan los datos.
- ☐ Se elige pago.
- ☐ Se muestra el resumen.
- ☐ Se crea la venta.
- ☐ El stock baja.
- ☐ La venta aparece en Angular.
- ☐ El doble clic no duplica.
- ☐ La tarjeta rechazada no crea venta.
- ☐ Sin caja se muestra un error claro.

Pedidos

- ☐ El pedido aparece en `/pedidos`.
- ☐ El detalle muestra los datos.
- ☐ El estado se actualiza.
- ☐ Confirmar en Angular se refleja.
- ☐ Anular en Angular se refleja.
- ☐ Otra sesión no ve el pedido.
- ☐ Volver a pedir funciona.

Calidad

- ☐ `npm run lint` pasa.
- ☐ `npx tsc --noEmit` pasa.
- ☐ `npm run build` pasa.
- ☐ No hay secretos en el bundle.
- ☐ No hay errores en consola.
- ☐ Funciona en móvil.
- ☐ Funciona con teclado.
- ☐ Los estados de error son claros.

---

24. Glosario

Término	Significado
BFF	Backend For Frontend; capa servidor de Next que habla con el backend
Venta	Registro real del backend; es lo que descuenta stock
Pedido	Vista de tienda sobre una venta + metadata temporal
Metadata	Datos que el backend no tiene: cliente, dirección, entrega, notas
Idempotencia	Evitar que un reintento cree otra venta
Caja	Turno abierto del POS; sin caja no se puede vender
UI status	Estado estimado que se muestra al cliente
Usuario técnico	Usuario de empleado que usa el BFF para hablar con el backend
OrderStore	Interfaz del almacén temporal de pedidos
Proxy	Nuevo nombre de middleware en Next.js 16

---

25. Referencias internas

- `AGENTS.md` del frontend: reglas del proyecto y lectura obligatoria de docs de Next 16.
- `node_modules/next/dist/docs/`: documentación local de Next 16.
- `docs/PLAN.md`: plan viejo de restaurante; no seguir.
- Backend `AGENTS.md`: estado real del backend, permisos y decisiones.
- Backend `SaleController.java`: endpoints y permisos de venta.
- Backend `SaleImpl.java`: caja, stock, idempotencia y pagos.
- Backend `ProductController.java`: catálogo y búsqueda.
- Backend `SecurityConfig.java`: prefijos, endpoints públicos y CORS.
- Backend `CashRegisterController.java`: caja activa.
- Frontend `app/layout.tsx`: layout actual.
- Frontend `app/globals.css`: Tailwind 4.
- Frontend `next.config.ts`: configuración pendiente.

---

26. Primer commit recomendado

Cuando se empiece a implementar, el primer commit debe ser pequeño y verificable:

```
chore: preparar tienda y cliente del backend
```

Incluye:

- `next.config.ts` con imágenes.
- `lib/server/env.ts`.
- `lib/server/backend/client.ts`.
- `lib/server/backend/auth.ts`.
- `app/api/bff/health/route.ts`.
- `.gitignore` con `.data/`.
- Sin UI nueva todavía.

Mensaje de verificación:

```
GET /api/bff/health responde:
{
  "backend": "ok",
  "auth": "ok",
  "cash": "open"
}
```

Ese es el punto de partida. Si eso no funciona, no tiene sentido construir el catálogo.

---

27. Conclusión

La tienda se puede construir sin tocar el backend porque el backend ya sabe vender. Lo que falta es una capa de tienda que:

- proteja el token;
- adapte los datos;
- valide en servidor;
- use idempotencia;
- guarde temporalmente lo que el backend no tiene;
- sea honesta sobre sus limitaciones.

El plan prioriza que el flujo real funcione antes que la apariencia. Primero se comprueba que una venta creada desde Next aparece en Angular y descuenta stock. Después se pule el diseño, el SEO y las pruebas.

La regla final:

> **Si una función no se puede sostener con el backend actual, no se simula como si existiera. Se documenta, se muestra como demo o se deja para la siguiente fase.**